

Quản lý Bug Report hiệu quả bằng GitHub

GitHub Issues · Labels · Projects · Pull Requests · Metrics

Software Testing

BUG → TRIAGE → FIX → RETEST → CLOSE

Sau buổi học, sinh viên làm được gì?

Kết quả cần đạt khi dùng GitHub để quản lý bug

- 1 Tạo bug report chuẩn**
Có steps, expected/actual, evidence, môi trường
- 2 Phân loại lỗi**
Dùng label type, severity, priority, module, status
- 3 Theo dõi xử lý**
Assignee, milestone, project board, workflow
- 4 Liên kết sửa lỗi**
Pull Request, commit, retest, close issue
- 5 Thống kê báo cáo**
Bugs by status/severity/priority/module/team

Kỹ năng chính
Quản lý lỗi có quy
trình
không chỉ “ghi lỗi”

Tại sao dùng GitHub Issues để quản lý bug?

Phù hợp cho nhóm nhỏ, lớp học, đồ án và dự án mã nguồn mở



Bug report

Ghi nhận lỗi có bằng chứng



Labels

Phân loại và lọc nhanh



Milestones

Gắn lỗi với sprint/release



Projects

Theo dõi bằng Kanban/board



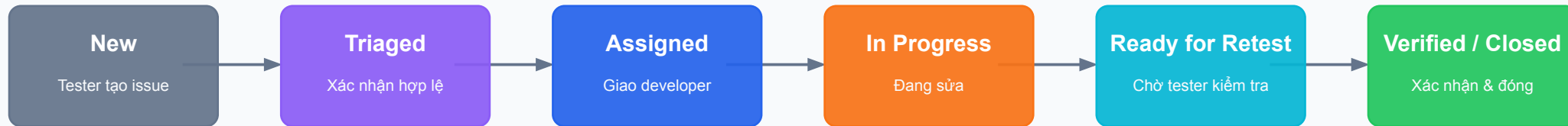
PR links

Nối bug với mã sửa lỗi

Nguyên tắc: một bug = một Issue = một luồng xử lý rõ ràng từ phát hiện → xác minh → sửa → kiểm thử lại.

Quy trình vòng đời một bug

Workflow đề xuất cho nhóm sinh viên



Nhánh ngoại lệ

Duplicate → đóng và link issue gốc

Cannot reproduce → yêu cầu thêm evidence

Won't fix → ghi rõ lý do nghiệp vụ/kỹ thuật

Reopened → bug vẫn còn sau khi retest

**Không đóng bug chỉ vì developer nói “đã sửa”.
Tester cần retest và comment kết quả.**

Một bug report tốt phải có gì?

Mẫu thông tin tối thiểu để lỗi có thể tái hiện và sửa được

Tiêu đề: [BUG][Checkout] User can place order with empty cart

Môi trường: Browser, OS, URL, version/commit, account test

Steps to reproduce: Các bước ngắn, đánh số, có dữ liệu đầu vào

Actual result: Hệ thống đang sai như thế nào?

Expected result: Hệ thống đúng phải hoạt động ra sao?

Evidence: Screenshot, video, console log, network response

Rõ ràng

Tái hiện được

Có bằng chứng

Có phân loại

Bug report càng cụ thể → Developer sửa nhanh hơn → Tester retest chính xác hơn.

Chuẩn hóa bằng Issue Template

Template giúp mọi sinh viên báo lỗi theo cùng một cấu trúc

```
---
name: Bug Report
title: "[BUG]: "
labels: ["type: bug", "status: new"]
---
```

```
## Mô tả lỗi
## Môi trường
## Steps to reproduce
1.
2.
3.
## Actual result
## Expected result
## Evidence
```

Cách tạo trong GitHub

Repository → Settings → Features → Issues

Chọn Set up templates

Tạo Bug report template

Có thể đặt label mặc định: type: bug, status: new

Có thể dùng Markdown template hoặc YAML Issue Form

Thiết kế hệ thống Labels

GitHub không có “nhóm label” như Jira, nên dùng quy ước tiền tố

TYPE

type: bug

type: enhancement

type: test

STATUS

status: new

status: triaged

status: in progress

status: ready for retest

status: verified

SEVERITY

severity: blocker

severity: critical

severity: major

severity: minor

severity: trivial

MODULE

module: login

module: cart

module: checkout

module: admin

module: api

PRIORITY

priority: P0

priority: P1

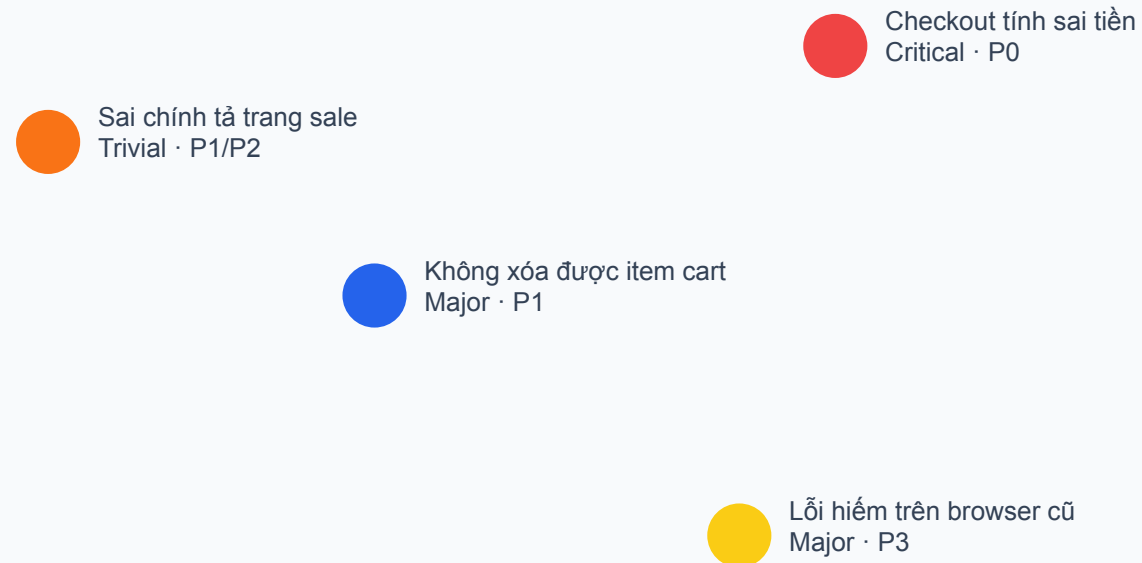
priority: P2

priority: P3

Severity khác Priority như thế nào?

Severity = mức độ nghiêm trọng; Priority = mức độ cần sửa gấp

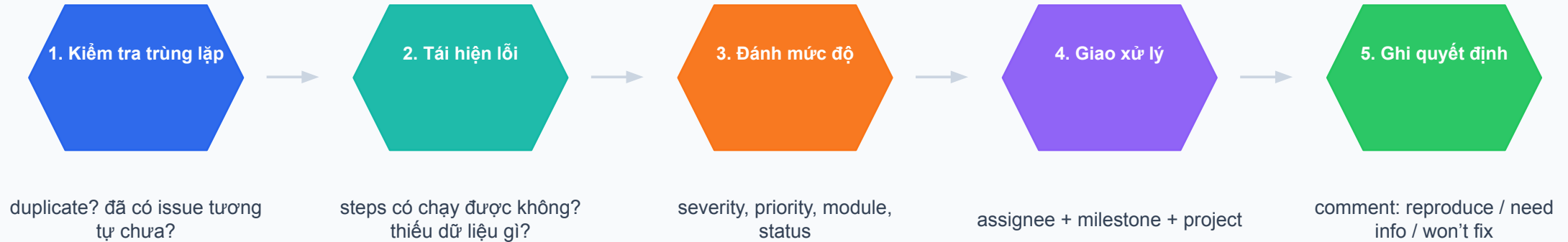
Priority cao



**Không phải bug nghiêm trọng nào cũng cần sửa ngay;
không phải bug nhỏ nào cũng được để sau.**

Bug Triage: lọc, xác nhận, ưu tiên

Không phải Issue nào cũng được giao sửa ngay



Kết quả triage hợp lệ

Issue có đủ thông tin để dev bắt đầu sửa

Có đúng label và mức ưu tiên

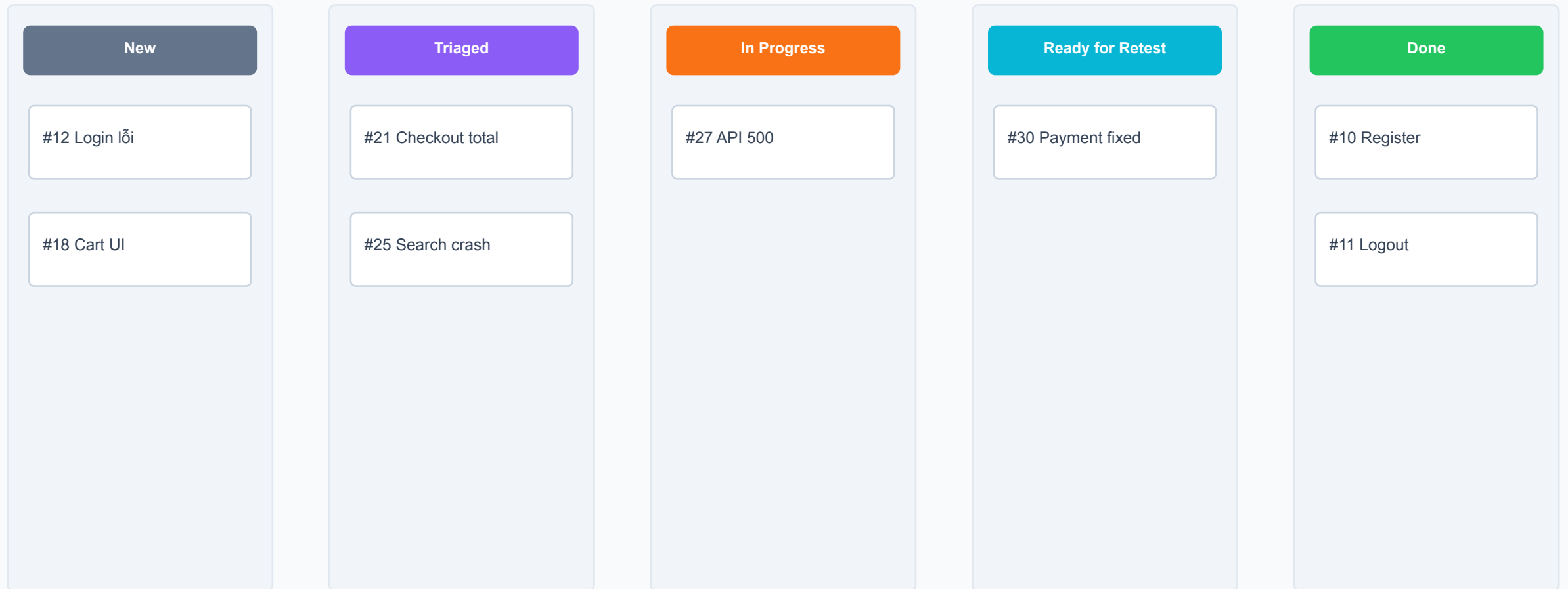
Có người phụ trách và mốc thời gian

Có quyết định rõ ràng nếu chưa sửa

Vai trò Test Lead: không sửa bug, nhưng làm cho bug “đủ rõ” để sửa.

GitHub Projects: quản lý bug như Kanban Board

Dùng để nhìn tiến độ thay vì chỉ nhìn danh sách Issue



Thao tác trong lớp: kéo thả Issue giữa các cột khi trạng thái thay đổi.

Milestone, Assignee và vai trò trong nhóm

Biến bug thành công việc có trách nhiệm và hạn xử lý

Milestone

Gom bug theo Sprint 1, Sprint 2, Release v1.0

Theo dõi tiến độ hoàn thành của một đợt test/release

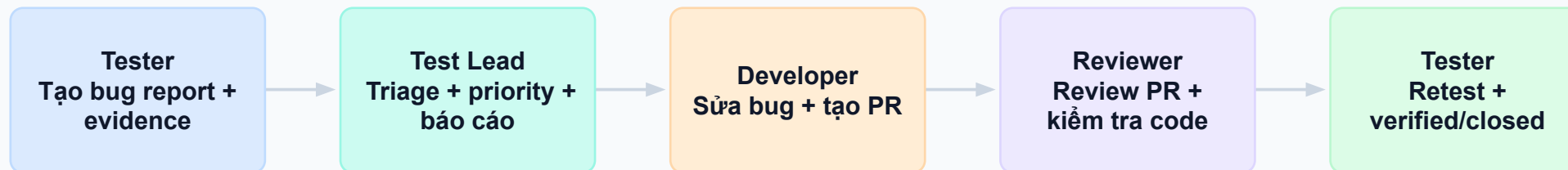
Dễ lọc: milestone:"Sprint 1"

Assignee

Gán người chịu trách nhiệm sửa hoặc xác minh

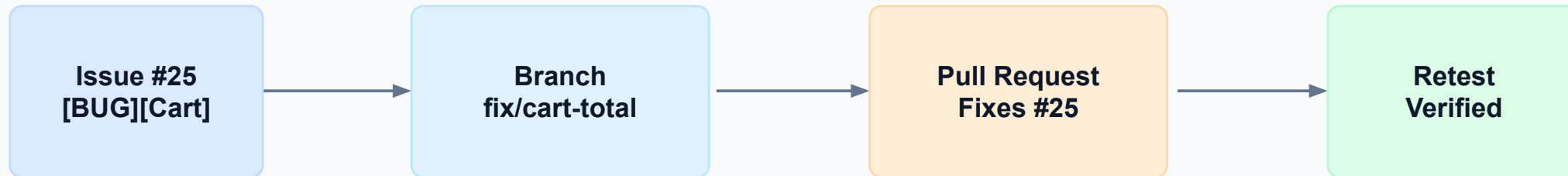
Một bug không nên “vô chủ” quá lâu

Dùng để cân bằng tải giữa các thành viên



Liên kết bug với Pull Request

Mỗi lỗi nên có dấu vết từ Issue → Commit/PR → Retest



Trong mô tả Pull Request: Fixes #25 hoặc Closes #25 hoặc Resolves #25

Trong quy trình có QA, nên chuyển sang Ready for Retest trước khi đóng hoàn toàn.

Thống kê nhanh bằng Issue Filters

Dùng query để đếm bug trực tiếp trong tab Issues

Tổng bug

```
is:issue label:"type: bug"
```

Bug đang mở

```
is:issue is:open label:"type: bug"
```

Bug nghiêm trọng

```
is:issue label:"severity: critical"
```

Bug cần sửa ngay

```
is:issue label:"priority: P0"
```

Bug module checkout

```
is:issue label:"module: checkout"
```

Bug của một sprint

```
is:issue label:"type: bug" milestone:"Sprint 1"
```

Tip: chuẩn hóa label càng tốt thì thống kê càng chính xác.

Bộ chỉ số bug nên báo cáo cuối sprint

Không chỉ đếm số bug, mà phải đọc được tình trạng chất lượng



Nhóm thống kê bắt buộc

Bugs by Status: New, In Progress, Ready for Retest, Done

Bugs by Severity: Blocker/Critical/Major/Minor/Trivial

Bugs by Priority: P0/P1/P2/P3

Bugs by Module: Login, Cart, Checkout, Admin, API, UI

Bugs by Assignee/Tester: phân bổ công việc và đóng góp

Câu hỏi phân tích
Module nào lỗi nhiều nhất?
Bug nghiêm trọng còn mở bao nhiêu?
Tốc độ fix có theo kịp tốc độ phát hiện không?

Xuất dữ liệu bug bằng GitHub CLI

Dùng khi cần báo cáo CSV/Excel hoặc xử lý bằng script

```
gh issue list \  
  --repo owner/repository \  
  --state all \  
  --label "type: bug" \  
  --limit 1000 \  
  --json number,title,state,labels,assignees,createdAt,closedAt \  
> bugs.json
```

Sau khi xuất dữ liệu

Dùng jq/Python để trích severity, priority, module từ labels

Mở trong Excel/Google Sheets để tạo Pivot Table

Tạo biểu đồ: bug theo module, theo trạng thái, theo người phụ trách

Nâng cao

**Dành cho báo cáo cuối kỳ
hoặc dashboard tự động.**

Lỗi thường gặp khi quản lý bug

Những điều cần tránh để GitHub Issues không trở thành “bãi rác”

❌ Tiêu đề mơ hồ

“Lỗi login”, “Bug giao diện”

❌ Thiếu bước tái hiện

Developer không biết làm sao để thấy lỗi

❌ Không có expected result

Không rõ hệ thống đúng phải ra sao

❌ Gắn label tùy tiện

Một issue có nhiều severity hoặc thiếu priority

❌ Đóng bug khi chưa retest

Không có comment verified từ tester

❌ Không thống kê cuối sprint

Không rút ra module rủi ro/chất lượng fix

Nguyên tắc kiểm soát: Issue nào không đủ thông tin thì không được đưa vào “In Progress”.

Checklist triển khai cho repository

- ✓ Bật Issues cho repository
- ✓ Tạo Bug Report Issue Template
- ✓ Tạo labels theo tiền tố: type, severity, priority, status, module
- ✓ Tạo milestone theo Sprint/Release
- ✓ Tạo GitHub Project board và field Status
- ✓ Quy định title format: [BUG][Module] Short description
- ✓ Quy định mỗi bug phải có evidence
- ✓ Quy định PR phải link Issue
- ✓ Quy định tester retest trước khi Close
- ✓ Cuối sprint xuất Bug Summary

Mục tiêu cuối cùng: mọi bug đều có người chịu trách nhiệm, trạng thái rõ ràng, bằng chứng đầy đủ và kết quả retest.